

Assignment 0

Shirou Jing · Rice ID: sj186

1 Anaconda

Anaconda3-2026.07-1-MacOSX-arm64.sh installed into ~/anaconda3 on macOS 15.1 (arm64), followed by `conda init zsh`. Result: conda 26.5.3 on Python 3.14.6, 552 packages in base. The full `conda list` is in the repository as `outputs/task1_conda_list.txt`.

Task 1 — conda info

```
(base) jsr.ovo._@ShiroutekiMacBook-Pro-2 ~ % conda info

active environment : base
active env location : /Users/jsr.ovo._/anaconda3
shell level : 1
user config file : /Users/jsr.ovo._/.condarc
populated config files : /Users/jsr.ovo._/anaconda3/.condarc
                        /Users/jsr.ovo._/anaconda3/condarc.d/anaconda-auth.yml
conda version : 26.5.3
conda-build version : 26.5.0
python version : 3.14.6.final.0
solver : libmamba (default)
virtual packages : __archspec=1=m4
                  __conda=26.5.3=0
                  __osx=15.1=0
                  __unix=0=0
base environment : /Users/jsr.ovo._/anaconda3 (writable)
conda av data dir : /Users/jsr.ovo._/anaconda3/etc/conda
conda av metadata url : None
channel URLs : https://repo.anaconda.com/pkgs/main/osx-arm64
               https://repo.anaconda.com/pkgs/main/noarch
               https://repo.anaconda.com/pkgs/r/osx-arm64
               https://repo.anaconda.com/pkgs/r/noarch
package cache : /Users/jsr.ovo._/anaconda3/pkgs
                /Users/jsr.ovo._/.conda/pkgs
envs directories : /Users/jsr.ovo._/anaconda3/envs
                  /Users/jsr.ovo._/.conda/envs
temporary directory : /var/folders/_d/tstckvp90tn2cjp23w7v85zw0000gn/T
platform : osx-arm64
user-agent : conda/26.5.3 requests/2.34.2 CPython/3.14.6 Darwin/24.1.0 OSX/15.1
1 solver/libmamba conda-libmamba-solver/26.7.0 libmambapy/2.3.2 aau/0.8.1 c/. s/. e/.
   UID:GID : 501:20
   netrc file : None
   offline mode : False
```

2 MATLAB to Python — Linear Algebra Equivalents

All 82 rows of the table were run in IPython, taking every spelling a row lists (both `np.ndim(a)` and `a.ndim`, for instance), for 161 cells. My matrices: `a`, `b`, `c`, `d` are 5×5 drawn uniformly from [0,1) and rounded to two decimals, `v` a matching 5-vector, seeded with `default_rng(576)` so the transcript reproduces exactly. Values in [0,1) keep the threshold rows (`a > 0.5`) non-trivial. Where several spellings in one row return the same array, the repeats are marked as identical rather than reprinted.

Rows that cannot be run exactly as printed

Row 28, 30, 76, 77 — assign or sort in place; `a` is restored from `a_orig` afterwards, shown in the transcript rather than done silently.

Row 46 — the table prints `np.ix_(np.r_[0:9.], np.r_[0:6.])` — a missing parenthesis; float indices also raise `IndexError`, so integer ranges are used.

Row 57, 58 — the bare `logical_and` is called as `np.logical_and`.

Row 59, 60 — bitwise operators need integers, so an integer pair `ai, bi` is used.

Row 65 — the table gives prose, not a command; written out as `linalg.solve(a.T, b.T).T`.

Row 67, 73 — Cholesky and conjugate gradients need positive definiteness, so `spd = a @ a.T + 5*np.eye(5)`.

Row 80 — `signal.resample` needs an integer sample count, so `int(np.ceil(...))`.

Row 82 — `a` has no singleton dimensions, so a $1 \times 3 \times 1 \times 4$ array is used instead.

Task 2 — IPython transcript

```
--- Setup -----
In [1]: import numpy as np
In [2]: import scipy.linalg
In [3]: from numpy.random import default_rng
In [4]: from scipy import signal
In [5]: from scipy.sparse.linalg import eigs, cg
In [6]: linalg = scipy.linalg          # the assignment asks us to import scipy.linalg
In [7]: np.set_printoptions(precision=4, suppress=True, linewidth=100)
In [8]: seed = default_rng(576)
In [9]: a = np.round(seed.random((5, 5)), 2)
In [10]: b = np.round(seed.random((5, 5)), 2)
In [11]: c = np.round(seed.random((5, 5)), 2)
In [12]: d = np.round(seed.random((5, 5)), 2)
In [13]: v = np.round(seed.random(5), 2)
In [14]: x = a.copy()
In [15]: m, n, q = 2, 2, 2
In [16]: ai = seed.integers(0, 8, (5, 5))          # integer arrays, for the bitwise rows
In [17]: bi = seed.integers(0, 8, (5, 5))
In [18]: spd = a @ a.T + 5 * np.eye(5)             # symmetric positive definite, for chol / cg
In [19]: sig = np.round(seed.random(12), 2)        # a 1-D signal, for resample
In [20]: a_orig = a.copy()                         # kept so the in-place rows can be undone
In [21]: a
Out[21]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12]])

In [22]: b
Out[22]:
array([[0.41, 0.38, 0.82, 0.08, 0.35],
       [0.58, 0.25, 0.54, 0.19, 0.72],
       [0.57, 0.09, 0.79, 0.43, 0.73],
       [0.94, 0.58, 0.54, 0.66, 0.5 ],
       [0.34, 0.41, 0.48, 0.97, 0.23]])

In [23]: v
Out[23]: array([0.66, 0.89, 0.71, 0.42, 0.78])

--- Row 1 --- MATLAB: ndims(a) -----
In [24]: np.ndim(a)
Out[24]: 2
In [25]: a.ndim
Out[25]: 2

--- Row 2 --- MATLAB: numel(a) -----
In [26]: np.size(a)
Out[26]: 25
In [27]: a.size
Out[27]: 25

--- Row 3 --- MATLAB: size(a) -----
In [28]: np.shape(a)
Out[28]: (5, 5)
In [29]: a.shape
Out[29]: (5, 5)

--- Row 4 --- MATLAB: size(a,n) -----
In [30]: a.shape[n-1]
```

```

Out[30]: 5

--- Row 5 --- MATLAB: [ 1 2 3; 4 5 6 ] -----
In [31]: np.array([[1., 2., 3.], [4., 5., 6.]])
Out[31]:
array([[1., 2., 3.],
       [4., 5., 6.]])

--- Row 6 --- MATLAB: [ a b; c d ] -----
In [32]: np.block([[a, b], [c, d]])
Out[32]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 , 0.41, 0.38, 0.82, 0.08, 0.35],
       [0.88, 0.3 , 0.44, 0.31, 0.28, 0.58, 0.25, 0.54, 0.19, 0.72],
       [0.54, 0.82, 0.43, 0.28, 0.86, 0.57, 0.09, 0.79, 0.43, 0.73],
       [0.54, 0.58, 0.88, 0.78, 0.21, 0.94, 0.58, 0.54, 0.66, 0.5 ],
       [0.2 , 0.8 , 0.66, 0.28, 0.12, 0.34, 0.41, 0.48, 0.97, 0.23],
       [0.71, 0.34, 0.46, 0.66, 0.89, 0.56, 0.76, 0.74, 0.42, 0.1 ],
       [0.28, 0.9 , 0.44, 0.87, 0.99, 0.39, 0.6 , 0.35, 0.86, 0.28],
       [0.7 , 0.9 , 0.72, 0.65, 0.82, 0.87, 0.32, 0.15, 0.86, 0.04],
       [0.06, 0.86, 0.66, 0.54, 0.1 , 0.35, 0.28, 0.72, 0.04, 0.58],
       [0.45, 0.71, 0.62, 0.88, 0.94, 0.17, 0.56, 0.6 , 0.25]])

--- Row 7 --- MATLAB: a(end) -----
In [33]: a[-1]
Out[33]: array([0.2 , 0.8 , 0.66, 0.28, 0.12])

--- Row 8 --- MATLAB: a(2,5) -----
In [34]: a[1, 4]
Out[34]: np.float64(0.28)

--- Row 9 --- MATLAB: a(2,:) -----
In [35]: a[1]
Out[35]: array([0.88, 0.3 , 0.44, 0.31, 0.28])
In [36]: a[1, : ]
Out[36]: array([0.88, 0.3 , 0.44, 0.31, 0.28])

--- Row 10 --- MATLAB: a(1:5,:) -----
In [37]: a[0:5]
Out[37]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12]])
In [38]: a[:5]
Out[38]: identical to Out[37]
In [39]: a[0:5, : ]
Out[39]: identical to Out[37]

--- Row 11 --- MATLAB: a(end-4:end,:) -----
In [40]: a[-5:]
Out[40]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12]])

--- Row 12 --- MATLAB: a(1:3,5:9) -----
note: a is 5x5, so columns 5..9 resolve to the single column 5.
In [41]: a[0:3, 4:9]
Out[41]:
array([[0.4 ],
       [0.28],
       [0.86]])

--- Row 13 --- MATLAB: a([2,4,5],[1,3]) -----
In [42]: a[np.ix_([1, 3, 4], [0, 2])]
Out[42]:
array([[0.88, 0.44],
       [0.54, 0.88],
       [0.2 , 0.66]])

--- Row 14 --- MATLAB: a(3:2:21,:) -----
In [43]: a[2:21:2, : ]
Out[43]:
array([[0.54, 0.82, 0.43, 0.28, 0.86],
       [0.2 , 0.8 , 0.66, 0.28, 0.12]])

--- Row 15 --- MATLAB: a(1:2:end,:) -----
In [44]: a[::2, : ]
Out[44]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.54, 0.82, 0.43, 0.28, 0.86],

```

```

[0.2 , 0.8 , 0.66, 0.28, 0.12]])

--- Row 16 --- MATLAB: a(end:-1:1,:) or flipud(a) -----
In [45]: a[::-1, :]
Out[45]:
array([[0.2 , 0.8 , 0.66, 0.28, 0.12],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.81, 0.86, 0.94, 0.06, 0.4 ]])

--- Row 17 --- MATLAB: a([1:end 1],:) -----
In [46]: a[np.r_[1:len(a), 0]]
Out[46]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12],
       [0.81, 0.86, 0.94, 0.06, 0.4 ]])

--- Row 18 --- MATLAB: a.' -----
In [47]: a.transpose()
Out[47]:
array([[0.81, 0.88, 0.54, 0.54, 0.2 ],
       [0.86, 0.3 , 0.82, 0.58, 0.8 ],
       [0.94, 0.44, 0.43, 0.88, 0.66],
       [0.06, 0.31, 0.28, 0.78, 0.28],
       [0.4 , 0.28, 0.86, 0.21, 0.12]])
In [48]: a.T
Out[48]: identical to Out[47]

--- Row 19 --- MATLAB: a' -----
In [49]: a.conj().transpose()
Out[49]:
array([[0.81, 0.88, 0.54, 0.54, 0.2 ],
       [0.86, 0.3 , 0.82, 0.58, 0.8 ],
       [0.94, 0.44, 0.43, 0.88, 0.66],
       [0.06, 0.31, 0.28, 0.78, 0.28],
       [0.4 , 0.28, 0.86, 0.21, 0.12]])
In [50]: a.conj().T
Out[50]: identical to Out[49]

--- Row 20 --- MATLAB: a * b -----
In [51]: a @ b
Out[51]:
array([[1.5591, 0.8062, 2.0956, 1.06 , 1.7109],
       [1.1722, 0.7436, 1.533 , 0.7928, 1.0646],
       [1.4977, 0.9639, 1.7893, 1.4029, 1.4311],
       [1.864 , 0.9679, 1.9732, 1.2503, 1.6873],
       [1.2262, 0.547 , 1.3262, 0.753 , 1.2954]])

--- Row 21 --- MATLAB: a .* b -----
In [52]: a * b
Out[52]:
array([[0.3321, 0.3268, 0.7708, 0.0048, 0.14 ],
       [0.5104, 0.075 , 0.2376, 0.0589, 0.2016],
       [0.3078, 0.0738, 0.3397, 0.1204, 0.6278],
       [0.5076, 0.3364, 0.4752, 0.5148, 0.105 ],
       [0.068 , 0.328 , 0.3168, 0.2716, 0.0276]])

--- Row 22 --- MATLAB: a./b -----
In [53]: a/b
Out[53]:
array([[1.9756, 2.2632, 1.1463, 0.75 , 1.1429],
       [1.5172, 1.2 , 0.8148, 1.6316, 0.3889],
       [0.9474, 9.1111, 0.5443, 0.6512, 1.1781],
       [0.5745, 1. , 1.6296, 1.1818, 0.42 ],
       [0.5882, 1.9512, 1.375 , 0.2887, 0.5217]])

--- Row 23 --- MATLAB: a.^3 -----
In [54]: a**3
Out[54]:
array([[0.5314, 0.6361, 0.8306, 0.0002, 0.064 ],
       [0.6815, 0.027 , 0.0852, 0.0298, 0.022 ],
       [0.1575, 0.5514, 0.0795, 0.022 , 0.6361],
       [0.1575, 0.1951, 0.6815, 0.4746, 0.0093],
       [0.008 , 0.512 , 0.2875, 0.022 , 0.0017]])

--- Row 24 --- MATLAB: (a > 0.5) -----
In [55]: (a > 0.5)
Out[55]:
array([[ True,  True,  True, False, False],

```

```

[ True, False, False, False, False],
[ True,  True, False, False,  True],
[ True,  True,  True,  True, False],
[False,  True,  True, False, False]])

--- Row 25 --- MATLAB: find(a > 0.5) -----
In [56]: np.nonzero(a > 0.5)
Out[56]:
(array([0, 0, 0, 1, 2, 2, 2, 3, 3, 3, 3, 4, 4]),
 array([0, 1, 2, 0, 0, 1, 4, 0, 1, 2, 3, 1, 2]))

--- Row 26 --- MATLAB: a[:,find(v > 0.5)] -----
In [57]: a[:, np.nonzero(v > 0.5)[0]]
Out[57]:
array([[0.81, 0.86, 0.94, 0.4 ],
       [0.88, 0.3 , 0.44, 0.28],
       [0.54, 0.82, 0.43, 0.86],
       [0.54, 0.58, 0.88, 0.21],
       [0.2 , 0.8 , 0.66, 0.12]])

--- Row 27 --- MATLAB: a[:,find(v>0.5)] -----
In [58]: a[:, v.T > 0.5]
Out[58]:
array([[0.81, 0.86, 0.94, 0.4 ],
       [0.88, 0.3 , 0.44, 0.28],
       [0.54, 0.82, 0.43, 0.86],
       [0.54, 0.58, 0.88, 0.21],
       [0.2 , 0.8 , 0.66, 0.12]])

--- Row 28 --- MATLAB: a(a<0.5)=0 -----
note: This row assigns in place, so a is restored afterwards.
In [59]: a[a < 0.5] = 0
In [60]: a
Out[60]:
array([[0.81, 0.86, 0.94, 0. , 0. ],
       [0.88, 0. , 0. , 0. , 0. ],
       [0.54, 0.82, 0. , 0. , 0.86],
       [0.54, 0.58, 0.88, 0.78, 0. ],
       [0. , 0.8 , 0.66, 0. , 0. ]])
In [61]: a = a_orig.copy() # undo, so later rows see the original a

--- Row 29 --- MATLAB: a .* (a>0.5) -----
In [62]: a * (a > 0.5)
Out[62]:
array([[0.81, 0.86, 0.94, 0. , 0. ],
       [0.88, 0. , 0. , 0. , 0. ],
       [0.54, 0.82, 0. , 0. , 0.86],
       [0.54, 0.58, 0.88, 0.78, 0. ],
       [0. , 0.8 , 0.66, 0. , 0. ]])

--- Row 30 --- MATLAB: a(:) = 3 -----
note: Also in place; a is restored afterwards.
In [63]: a[:] = 3
In [64]: a
Out[64]:
array([[3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3.]])
In [65]: a = a_orig.copy() # undo again

--- Row 31 --- MATLAB: y=x -----
In [66]: y = x.copy()
In [67]: y
Out[67]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12]])

--- Row 32 --- MATLAB: y=x(2,:) -----
In [68]: y = x[1, :].copy()
In [69]: y
Out[69]: array([0.88, 0.3 , 0.44, 0.31, 0.28])

--- Row 33 --- MATLAB: y=x(:) -----
In [70]: y = x.flatten()
In [71]: y
Out[71]:
array([0.81, 0.86, 0.94, 0.06, 0.4 , 0.88, 0.3 , 0.44, 0.31, 0.28, 0.54, 0.82, 0.43, 0.28, 0.86,
       0.54, 0.58, 0.88, 0.78, 0.21, 0.2 , 0.8 , 0.66, 0.28, 0.12])

```

```

--- Row 34 --- MATLAB: 1:10 -----
In [72]: np.arange(1., 11.)
Out[72]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
In [73]: np.r_[1.:11.]
Out[73]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
In [74]: np.r_[1:10:10j]
Out[74]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])

--- Row 35 --- MATLAB: 0:9 -----
In [75]: np.arange(10.)
Out[75]: array([0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.])
In [76]: np.r_[1:10.]
Out[76]: array([0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.])
In [77]: np.r_[9:10j]
Out[77]: array([0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.])

--- Row 36 --- MATLAB: [1:10]' -----
In [78]: np.arange(1., 11.)[:, np.newaxis]
Out[78]:
array([[ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.],
       [ 6.],
       [ 7.],
       [ 8.],
       [ 9.],
       [10.]])

--- Row 37 --- MATLAB: zeros(3,4) -----
In [79]: np.zeros((3, 4))
Out[79]:
array([[0.,  0.,  0.,  0.],
       [0.,  0.,  0.,  0.],
       [0.,  0.,  0.,  0.]])

--- Row 38 --- MATLAB: zeros(3,4,5) -----
In [80]: np.zeros((3, 4, 5))
Out[80]:
array([[[0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.]],

       [[0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.]],

       [[0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.],
        [0.,  0.,  0.,  0.,  0.]])

--- Row 39 --- MATLAB: ones(3,4) -----
In [81]: np.ones((3, 4))
Out[81]:
array([[1.,  1.,  1.,  1.],
       [1.,  1.,  1.,  1.],
       [1.,  1.,  1.,  1.]])

--- Row 40 --- MATLAB: eye(3) -----
In [82]: np.eye(3)
Out[82]:
array([[1.,  0.,  0.],
       [0.,  1.,  0.],
       [0.,  0.,  1.]])

--- Row 41 --- MATLAB: diag(a) -----
In [83]: np.diag(a)
Out[83]: array([0.81, 0.3 , 0.43, 0.78, 0.12])

--- Row 42 --- MATLAB: diag(v,0) -----
In [84]: np.diag(v, 0)
Out[84]:
array([[0.66, 0. , 0. , 0. , 0. ],
       [0. , 0.89, 0. , 0. , 0. ],
       [0. , 0. , 0.71, 0. , 0. ],
       [0. , 0. , 0. , 0.42, 0. ],
       [0. , 0. , 0. , 0. , 0.78]])

--- Row 43 --- MATLAB: rng(42,'twister'); rand(3,4) -----

```

```

In [85]: from numpy.random import default_rng
In [86]: rng = default_rng(42)
In [87]: rng.random((3, 4))
Out[87]:
array([[0.774 , 0.4389, 0.8586, 0.6974],
       [0.0942, 0.9756, 0.7611, 0.7861],
       [0.1281, 0.4504, 0.3708, 0.9268]])

--- Row 44 --- MATLAB: linspace(1,3,4) -----
In [88]: np.linspace(1, 3, 4)
Out[88]: array([1.      , 1.6667, 2.3333, 3.      ])

--- Row 45 --- MATLAB: [x,y]=meshgrid(0:8,0:5) -----
In [89]: np.mgrid[0:9., 0:6.]
Out[89]:
array([[0., 0., 0., 0., 0., 0.],
       [1., 1., 1., 1., 1., 1.],
       [2., 2., 2., 2., 2., 2.],
       [3., 3., 3., 3., 3., 3.],
       [4., 4., 4., 4., 4., 4.],
       [5., 5., 5., 5., 5., 5.],
       [6., 6., 6., 6., 6., 6.],
       [7., 7., 7., 7., 7., 7.],
       [8., 8., 8., 8., 8., 8.]],

       [[0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.],
       [0., 1., 2., 3., 4., 5.]])
In [90]: np.meshgrid(np.r_[0:9.], np.r_[0:6.])
Out[90]:
(array([[0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.],
       [0., 1., 2., 3., 4., 5., 6., 7., 8.]]),
 array([[0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [2., 2., 2., 2., 2., 2., 2., 2., 2.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [5., 5., 5., 5., 5., 5., 5., 5., 5.]])

--- Row 46 --- MATLAB: (best way to eval functions on a grid) -----
note: The table prints np.ix_(np.r_[0:9.],np.r_[0:6.] - a missing
      paren, and float indices raise IndexError. Integer ranges are
      used here.
In [91]: np.ogrid[0:9., 0:6.]
Out[91]:
(array([[0.],
       [1.],
       [2.],
       [3.],
       [4.],
       [5.],
       [6.],
       [7.],
       [8.]]),
 array([[0., 1., 2., 3., 4., 5.]])
In [92]: np.ix_(np.r_[0:9.], np.r_[0:6.])
Out[92]:
(array([[0.],
       [1.],
       [2.],
       [3.],
       [4.],
       [5.],
       [6.],
       [7.],
       [8.]]),
 array([[0, 1, 2, 3, 4, 5]]))

--- Row 47 --- MATLAB: [x,y]=meshgrid([1,2,4],[2,4,5]) -----
In [93]: np.meshgrid([1, 2, 4], [2, 4, 5])
Out[93]:
(array([[1, 2, 4],

```

```

        [1, 2, 4],
        [1, 2, 4]]),
array([[2, 2, 2],
       [4, 4, 4],
       [5, 5, 5]])

--- Row 48 --- MATLAB: (best way to eval functions on a grid) -----
In [94]: np.ix_([1, 2, 4], [2, 4, 5])
Out[94]:
(array([[1],
        [2],
        [4]]),
 array([[2, 4, 5]]))

--- Row 49 --- MATLAB: repmat(a, m, n) -----
In [95]: np.tile(a, (m, n))
Out[95]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 , 0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28, 0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86, 0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21, 0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12, 0.2 , 0.8 , 0.66, 0.28, 0.12],
       [0.81, 0.86, 0.94, 0.06, 0.4 , 0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28, 0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86, 0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21, 0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12, 0.2 , 0.8 , 0.66, 0.28, 0.12]])

--- Row 50 --- MATLAB: [a b] -----
In [96]: np.concatenate((a, b), 1)
Out[96]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 , 0.41, 0.38, 0.82, 0.08, 0.35],
       [0.88, 0.3 , 0.44, 0.31, 0.28, 0.58, 0.25, 0.54, 0.19, 0.72],
       [0.54, 0.82, 0.43, 0.28, 0.86, 0.57, 0.09, 0.79, 0.43, 0.73],
       [0.54, 0.58, 0.88, 0.78, 0.21, 0.94, 0.58, 0.54, 0.66, 0.5 ],
       [0.2 , 0.8 , 0.66, 0.28, 0.12, 0.34, 0.41, 0.48, 0.97, 0.23]])

In [97]: np.hstack((a, b))
Out[97]: identical to Out[96]
In [98]: np.column_stack((a, b))
Out[98]: identical to Out[96]
In [99]: np.c_[a, b]
Out[99]: identical to Out[96]

--- Row 51 --- MATLAB: [a; b] -----
In [100]: np.concatenate((a, b))
Out[100]:
array([[0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.2 , 0.8 , 0.66, 0.28, 0.12],
       [0.41, 0.38, 0.82, 0.08, 0.35],
       [0.58, 0.25, 0.54, 0.19, 0.72],
       [0.57, 0.09, 0.79, 0.43, 0.73],
       [0.94, 0.58, 0.54, 0.66, 0.5 ],
       [0.34, 0.41, 0.48, 0.97, 0.23]])

In [101]: np.vstack((a, b))
Out[101]: identical to Out[100]
In [102]: np.r_[a, b]
Out[102]: identical to Out[100]

--- Row 52 --- MATLAB: max(max(a)) -----
In [103]: a.max()
Out[103]: np.float64(0.94)
In [104]: np.nanmax(a)
Out[104]: np.float64(0.94)

--- Row 53 --- MATLAB: max(a) -----
In [105]: a.max(0)
Out[105]: array([0.88, 0.86, 0.94, 0.78, 0.86])

--- Row 54 --- MATLAB: max(a, [], 2) -----
In [106]: a.max(1)
Out[106]: array([0.94, 0.88, 0.86, 0.88, 0.8 ])

--- Row 55 --- MATLAB: max(a,b) -----
In [107]: np.maximum(a, b)
Out[107]:
array([[0.81, 0.86, 0.94, 0.08, 0.4 ],
       [0.88, 0.3 , 0.54, 0.31, 0.72],
       [0.57, 0.82, 0.79, 0.43, 0.86],
       [0.94, 0.58, 0.88, 0.78, 0.5 ],
       [0.34, 0.8 , 0.66, 0.97, 0.23]])

```

```

--- Row 56 --- MATLAB: norm(v) -----
In [108]: np.sqrt(v @ v)
Out[108]: np.float64(1.586379525838631)
In [109]: np.linalg.norm(v)
Out[109]: np.float64(1.586379525838631)

--- Row 57 --- MATLAB: a & b -----
note: The table writes a bare logical_and; it lives in the np
      namespace.
In [110]: np.logical_and(a, b)
Out[110]:
array([[ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True]])

--- Row 58 --- MATLAB: a | b -----
In [111]: np.logical_or(a, b)
Out[111]:
array([[ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True]])

--- Row 59 --- MATLAB: bitand(a,b) -----
note: Bitwise operators need integer arrays, so the integer pair ai,
      bi is used.
In [112]: ai & bi
Out[112]:
array([[2, 2, 0, 0, 0],
       [0, 4, 6, 4, 6],
       [0, 4, 6, 0, 5],
       [2, 0, 0, 0, 4],
       [1, 2, 7, 0, 1]])

--- Row 60 --- MATLAB: bitor(a,b) -----
note: Same as row 59.
In [113]: ai | bi
Out[113]:
array([[2, 7, 7, 0, 3],
       [3, 5, 7, 7, 6],
       [1, 7, 6, 7, 7],
       [7, 0, 6, 4, 5],
       [7, 2, 7, 7, 7]])

--- Row 61 --- MATLAB: inv(a) -----
In [114]: linalg.inv(a)
Out[114]:
array([[ -0.5073,  2.2723, -0.3672, -1.0236,  0.8117],
       [ -1.5891,  1.7277,  0.1441, -1.8824,  3.5275],
       [  2.6014, -2.7594, -0.4208,  2.0618, -2.8253],
       [ -1.6941,  0.7799,  0.2488,  0.7514,  0.7288],
       [  1.0846, -1.9483,  1.3854,  1.162 , -2.6978]])

--- Row 62 --- MATLAB: pinv(a) -----
In [115]: linalg.pinv(a)
Out[115]:
array([[ -0.5073,  2.2723, -0.3672, -1.0236,  0.8117],
       [ -1.5891,  1.7277,  0.1441, -1.8824,  3.5275],
       [  2.6014, -2.7594, -0.4208,  2.0618, -2.8253],
       [ -1.6941,  0.7799,  0.2488,  0.7514,  0.7288],
       [  1.0846, -1.9483,  1.3854,  1.162 , -2.6978]])

--- Row 63 --- MATLAB: rank(a) -----
In [116]: np.linalg.matrix_rank(a)
Out[116]: np.int64(5)

--- Row 64 --- MATLAB: a\b -----
In [117]: linalg.solve(a, b)
Out[117]:
array([[ 0.2144,  0.0813,  0.3578,  0.345 ,  0.8653],
       [-0.1375,  0.1955,  0.4204,  2.4424,  0.663 ],
       [ 0.2038,  0.2983,  0.0679, -1.8768, -1.0023],
       [ 0.8538,  0.3083, -0.0158,  1.3226,  0.6936],
       [ 0.2794, -0.3824,  0.2643, -1.5376, -0.0514]])
In [118]: linalg.lstsq(a, b)
Out[118]:
(array([[ 0.2144,  0.0813,  0.3578,  0.345 ,  0.8653],
       [-0.1375,  0.1955,  0.4204,  2.4424,  0.663 ],
       [ 0.2038,  0.2983,  0.0679, -1.8768, -1.0023],
       [ 0.8538,  0.3083, -0.0158,  1.3226,  0.6936],
       [ 0.2794, -0.3824,  0.2643, -1.5376, -0.0514]]))

```

```

    [ 0.2794, -0.3824, 0.2643, -1.5376, -0.0514]],
    array([], dtype=float64),
    np.int64(5),
    array([2.7835, 0.7063, 0.6235, 0.5049, 0.1221]))

--- Row 65 --- MATLAB: b/a -----
note: The table says 'Solve a.T x.T = b.T instead'; that is written
      out here.
In [119]: linalg.solve(a.T, b.T).T
Out[119]:
array([[ 1.5654, -1.2941, 0.0639, 1.0225, -1.5294],
       [ 1.1723, -0.9949, 0.6406, 1.0285, -1.9769],
       [ 1.6862, -1.8162, 0.5896, 2.0474, -3.1078],
       [-0.5696, 1.1885, 0.3681, 0.1363, 0.4155],
       [-0.9692, 0.4648, 0.2923, 0.866 , 0.4526]])

--- Row 66 --- MATLAB: [U,S,V]=svd(a) -----
In [120]: U, S, Vh = linalg.svd(a); V = Vh.T
In [121]: U
Out[121]:
array([[ -0.5446, -0.1601, -0.0456, -0.7099, 0.4144],
       [-0.3588, -0.0775, 0.764 , -0.0638, -0.5268],
       [-0.4618, -0.6677, -0.2098, 0.5433, 0.0426],
       [-0.4796, 0.6603, 0.1312, 0.4256, 0.3682],
       [-0.3624, 0.2942, -0.5942, -0.1255, -0.6429]])
In [122]: S
Out[122]: array([2.7835, 0.7063, 0.6235, 0.5049, 0.1221])
In [123]: V
Out[123]:
array([[ -0.4806, -0.2025, 0.7604, -0.2636, -0.2834],
       [-0.5471, -0.1276, -0.6115, -0.0747, -0.5522],
       [-0.5495, 0.4297, -0.118 , -0.3368, 0.6213],
       [-0.269 , 0.5335, 0.1785, 0.7656, -0.158 ],
       [-0.3088, -0.6881, -0.0457, 0.4747, 0.4513]])

--- Row 67 --- MATLAB: chol(a) -----
note: Cholesky needs a positive definite matrix, so spd = a @ a.T + 5I
      is used.
In [124]: linalg.cholesky(spd)
Out[124]:
array([[2.7282, 0.5553, 0.6992, 0.6943, 0.5627],
       [0. , 2.434 , 0.3491, 0.3909, 0.2113],
       [0. , 0. , 2.5211, 0.366 , 0.3023],
       [0. , 0. , 0. , 2.5072, 0.324 ],
       [0. , 0. , 0. , 0. , 2.3771]])

--- Row 68 --- MATLAB: [V,D]=eig(a) -----
In [125]: D, V = linalg.eig(a)
In [126]: D
Out[126]: array([ 2.652 +0.j , -0.6227+0.j , 0.5534+0.j , -0.0713+0.2786j, -0.0713-0.2786j])
In [127]: V
Out[127]:
array([[ 0.5051+0.j , 0.3539+0.j , -0.4778+0.j , -0.1411-0.2991j, -0.1411+0.2991j],
       [ 0.3841+0.j , -0.384 +0.j , -0.1696+0.j , -0.5436+0.1455j, -0.5436-0.1455j],
       [ 0.4611+0.j , -0.485 +0.j , 0.1419+0.j , 0.5901+0.j , 0.5901-0.j ],
       [ 0.5195+0.j , 0.2275+0.j , 0.8228+0.j , -0.2182-0.0436j, -0.2182+0.0436j],
       [ 0.3389+0.j , 0.6636+0.j , 0.2142+0.j , 0.334 +0.2544j, 0.334 -0.2544j]])

--- Row 69 --- MATLAB: [V,D]=eig(a,b) -----
In [128]: D, V = linalg.eig(a, b)
In [129]: D
Out[129]: array([-2.0252+0.j , -1.3202+0.j , 1.5696+0.j , 0.4611+0.4215j, 0.4611-0.4215j])
In [130]: V
Out[130]:
array([[ 0.0841+0.j , -0.1273+0.j , 0.0112+0.j , 0.2637+0.1323j, 0.2637-0.1323j],
       [-0.7583+0.j , -0.7367+0.j , -0.6623+0.j , -0.3473+0.3557j, -0.3473-0.3557j],
       [ 0.5327+0.j , 0.6178+0.j , -0.6383+0.j , 0.0315-0.3647j, 0.0315+0.3647j],
       [ 0.2088+0.j , 0.2082+0.j , -0.1113+0.j , -0.1162+0.4314j, -0.1162-0.4314j],
       [-0.301 +0.j , -0.1263+0.j , 0.3761+0.j , 0.5258-0.236j , 0.5258+0.236j ]])

--- Row 70 --- MATLAB: [V,D]=eigs(a,3) -----
note: eigs comes from scipy.sparse.linalg and needs k < n-1.
In [131]: D, V = eigs(a, k=3)
In [132]: D
Out[132]: array([ 2.652 +0.j, 0.5534+0.j, -0.6227+0.j])
In [133]: V
Out[133]:
array([[ -0.5051+0.j, -0.4778+0.j, -0.3539+0.j],
       [-0.3841+0.j, -0.1696+0.j, 0.384 +0.j],
       [-0.4611+0.j, 0.1419+0.j, 0.485 +0.j],
       [-0.5195+0.j, 0.8228+0.j, -0.2275+0.j],
       [-0.3389+0.j, 0.2142+0.j, -0.6636+0.j]])

```

```

--- Row 71 --- MATLAB: [Q,R]=qr(a,0) -----
In [134]: Q, R = linalg.qr(a)
In [135]: Q
Out[135]:
array([[ -0.5652,  0.1372,  0.1677,  0.7469, -0.2753],
       [ -0.6141, -0.5769, -0.1607, -0.1404,  0.4946],
       [ -0.3768,  0.3737, -0.6986, -0.3266, -0.3517],
       [ -0.3768,  0.0991,  0.6674, -0.5619, -0.295 ],
       [ -0.1396,  0.7063,  0.1122, -0.0081,  0.6848]])

In [136]: R
Out[136]:
array([[ -1.4331, -1.3095, -1.3872, -0.6628, -0.818 ],
       [  0.      ,  0.8739,  0.5892,  0.2091,  0.3203],
       [  0.      ,  0.      ,  0.4479,  0.3166, -0.4251],
       [  0.      ,  0.      ,  0.      , -0.5307, -0.1404],
       [  0.      ,  0.      ,  0.      ,  0.      , -0.2538]])

--- Row 72 --- MATLAB: [L,U,P]=lu(a) -----
In [137]: P, L, U = linalg.lu(a)
In [138]: P
Out[138]:
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 1.],
       [0., 1., 0., 0., 0.]])

In [139]: L
Out[139]:
array([[ 1.      ,  0.      ,  0.      ,  0.      ,  0.      ],
       [ 0.2273,  1.      ,  0.      ,  0.      ,  0.      ],
       [ 0.6136,  0.8689,  1.      ,  0.      ,  0.      ],
       [ 0.9205,  0.7978, -0.2701,  1.      ,  0.      ],
       [ 0.6136,  0.541 , -0.9401, -0.9333,  1.      ]])

In [140]: U
Out[140]:
array([[ 0.88 ,  0.3 ,  0.44 ,  0.31 ,  0.28 ],
       [ 0.   ,  0.7318,  0.56 ,  0.2095,  0.0564],
       [ 0.   ,  0.   , -0.3266, -0.0923,  0.6392],
       [ 0.   ,  0.   ,  0.   , -0.4175,  0.27  ],
       [ 0.   ,  0.   ,  0.   ,  0.   ,  0.8606]])

In [141]: np.allclose(a, P @ L @ U)
Out[141]: True

--- Row 73 --- MATLAB: conjgrad -----
note: cg is scipy's conjugate gradient solver; it needs an SPD system.
In [142]: cg(spd, v)
Out[142]: (array([ 0.0327,  0.1116,  0.0572, -0.001 ,  0.0916]), 0)

--- Row 74 --- MATLAB: fft(a) -----
In [143]: np.fft.fft(a)
Out[143]:
array([[ 3.07 +0.j ,  0.3903-0.9547j,  0.0997+0.5665j,  0.0997-0.5665j,  0.3903+0.9547j],
       [ 2.21 +0.j ,  0.4525-0.0954j,  0.6425+0.1119j,  0.6425-0.1119j,  0.4525+0.0954j],
       [ 2.93 +0.j ,  0.4847-0.0501j, -0.5997+0.1662j, -0.5997-0.1662j,  0.4847+0.0501j],
       [ 2.99 +0.j , -0.5588-0.4107j,  0.4138-0.1224j,  0.4138+0.1224j, -0.5588+0.4107j],
       [ 2.06 +0.j , -0.2762-0.8701j, -0.2538-0.0383j, -0.2538+0.0383j, -0.2762+0.8701j]])

--- Row 75 --- MATLAB: ifft(a) -----
In [144]: np.fft.ifft(a)
Out[144]:
array([[ 0.614 +0.j ,  0.0781+0.1909j,  0.0199-0.1133j,  0.0199+0.1133j,  0.0781-0.1909j],
       [ 0.442 +0.j ,  0.0905+0.0191j,  0.1285-0.0224j,  0.1285+0.0224j,  0.0905-0.0191j],
       [ 0.586 +0.j ,  0.0969+0.01j , -0.1199-0.0332j, -0.1199+0.0332j,  0.0969-0.01j ],
       [ 0.598 +0.j , -0.1118+0.0821j,  0.0828+0.0245j,  0.0828-0.0245j, -0.1118-0.0821j],
       [ 0.412 +0.j , -0.0552+0.174j , -0.0508+0.0077j, -0.0508-0.0077j, -0.0552-0.174j ]])

--- Row 76 --- MATLAB: sort(a) -----
In [145]: np.sort(a)
Out[145]:
array([[0.06, 0.4 , 0.81, 0.86, 0.94],
       [0.28, 0.3 , 0.31, 0.44, 0.88],
       [0.28, 0.43, 0.54, 0.82, 0.86],
       [0.21, 0.54, 0.58, 0.78, 0.88],
       [0.12, 0.2 , 0.28, 0.66, 0.8 ]])

In [146]: a.sort(axis=0); a
Out[146]:
array([[0.2 , 0.3 , 0.43, 0.06, 0.12],
       [0.54, 0.58, 0.44, 0.28, 0.21],
       [0.54, 0.8 , 0.66, 0.28, 0.28],
       [0.81, 0.82, 0.88, 0.31, 0.4 ],
       [0.88, 0.86, 0.94, 0.78, 0.86]])

In [147]: a = a_orig.copy() # sort is in place

```

```

--- Row 77 --- MATLAB: sort(a, 2) -----
In [148]: np.sort(a, axis=1)
Out[148]:
array([[0.06, 0.4 , 0.81, 0.86, 0.94],
       [0.28, 0.3 , 0.31, 0.44, 0.88],
       [0.28, 0.43, 0.54, 0.82, 0.86],
       [0.21, 0.54, 0.58, 0.78, 0.88],
       [0.12, 0.2 , 0.28, 0.66, 0.8 ]])
In [149]: a.sort(axis=1); a
Out[149]: identical to Out[148]
In [150]: a = a_orig.copy()

--- Row 78 --- MATLAB: [b,I]=sortrows(a,1) -----
In [151]: I = np.argsort(a[:, 0]); b = a[I, :]
In [152]: I
Out[152]: array([4, 2, 3, 0, 1])
In [153]: b
Out[153]:
array([[0.2 , 0.8 , 0.66, 0.28, 0.12],
       [0.54, 0.82, 0.43, 0.28, 0.86],
       [0.54, 0.58, 0.88, 0.78, 0.21],
       [0.81, 0.86, 0.94, 0.06, 0.4 ],
       [0.88, 0.3 , 0.44, 0.31, 0.28]])

--- Row 79 --- MATLAB: x = Z\y -----
In [154]: Z = a; y = v # rebind, rows 31-33 reassigned x and y
In [155]: x = linalg.lstsq(Z, y)
In [156]: x
Out[156]:
(array([ 1.63 , 2.552 , -2.3754, 0.6368, -1.6508]),
 array([], dtype=float64),
 np.int64(5),
 array([2.7835, 0.7063, 0.6235, 0.5049, 0.1221]))

--- Row 80 --- MATLAB: decimate(x, q) -----
note: np.ceil returns a float; resample needs an int, so int(...) is
      applied.
In [157]: sig
Out[157]: array([0.1 , 0.97, 0.58, 0.35, 0.15, 0.62, 0.88, 0.44, 0.48, 0.32, 0.47, 0.98])
In [158]: signal.resample(sig, int(np.ceil(len(sig)/q)))
Out[158]: array([0.5243, 0.6842, 0.1882, 0.8574, 0.2725, 0.6435])

--- Row 81 --- MATLAB: unique(a) -----
In [159]: np.unique(a)
Out[159]:
array([0.06, 0.12, 0.2 , 0.21, 0.28, 0.3 , 0.31, 0.4 , 0.43, 0.44, 0.54, 0.58, 0.66, 0.78, 0.8 ,
       0.81, 0.82, 0.86, 0.88, 0.94])

--- Row 82 --- MATLAB: squeeze(a) -----
note: a has no singleton dimensions, so a 1x3x1x4 array is used to
      show the effect.
In [160]: np.zeros((1, 3, 1, 4)).squeeze()
Out[160]:
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
In [161]: np.zeros((1, 3, 1, 4)).squeeze().shape
Out[161]: (3, 4)

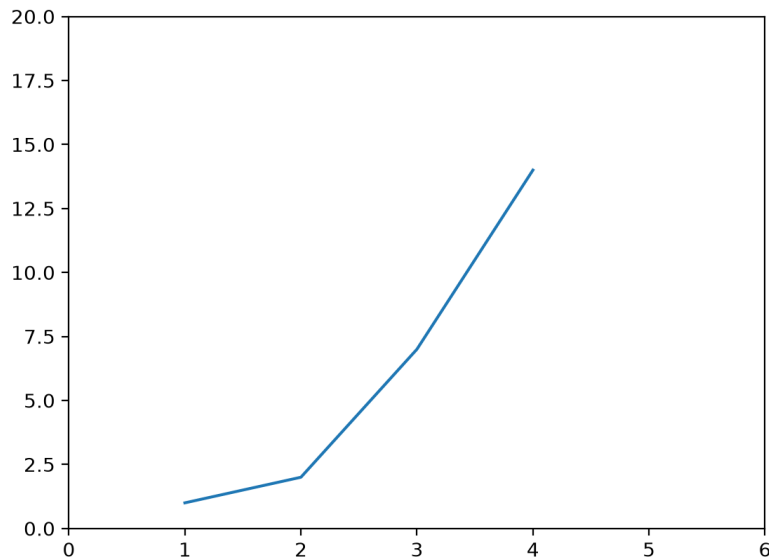
--- End of Task 2: 161 cells, all 82 table rows covered. -----

```

3 Plotting — the given script

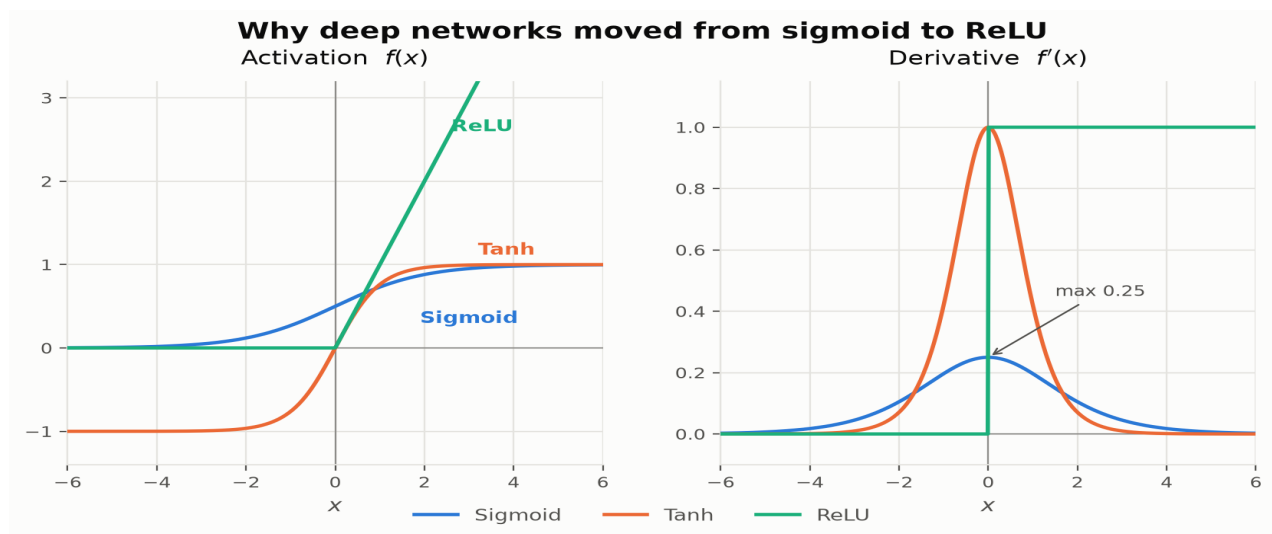
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4], [1,2,7,14])
plt.axis([0, 6, 0, 20])
plt.show()
```

Run verbatim in IPython. To capture it, the script was run under the Agg backend with a trailing `plt.savefig;` under Agg, `plt.show()` warns that it cannot display interactively, which does not affect the figure.



4 Plotting — a figure of my own

The three classical activation functions against their derivatives. The right panel is the point: sigmoid's derivative peaks at 0.25 and decays to zero in both tails, so a product of many such factors vanishes with depth, while ReLU's is exactly 1 on the positive half-line.



```
"""
ELEC 576 / COMP 576 - Assignment 0, Task 4
A figure of my own: three activation functions and their derivatives.
```

```
The right-hand panel is the point of the figure - sigmoid's gradient peaks at
0.25 and decays to zero in both tails, which is the vanishing-gradient problem
```

```

that motivates ReLU in deep networks.
"""
import matplotlib
matplotlib.use("Agg")
import numpy as np
import matplotlib.pyplot as plt

SURFACE = "#fcfcfb"
INK      = "#0b0b0b"
MUTED    = "#52514e"
GRID     = "#e4e3df"
SERIES   = ["#2a78d6", "#eb6834", "#1baf7a"]      # blue, orange, aqua

x = np.linspace(-6, 6, 600)

sigmoid = 1 / (1 + np.exp(-x))
tanh     = np.tanh(x)
relu     = np.maximum(0, x)

d_sigmoid = sigmoid * (1 - sigmoid)
d_tanh    = 1 - tanh**2
d_relu    = (x > 0).astype(float)

names = ["Sigmoid", "Tanh", "ReLU"]
funcs = [sigmoid, tanh, relu]
derivs = [d_sigmoid, d_tanh, d_relu]

fig, axes = plt.subplots(1, 2, figsize=(10, 4.2), facecolor=SURFACE)
fig.subplots_adjust(wspace=0.22)

for ax, ys, title, ylim in [
    (axes[0], funcs, "Activation   $f(x)$ ", (-1.4, 3.2)),
    (axes[1], derivs, "Derivative  $f'(\prime)(x)$ ", (-0.1, 1.15)),
]:
    ax.set_facecolor(SURFACE)
    ax.grid(True, color=GRID, linewidth=0.8, zorder=0)
    ax.set_axisbelow(True)
    ax.axhline(0, color=MUTED, linewidth=0.8, alpha=0.6, zorder=1)
    ax.axvline(0, color=MUTED, linewidth=0.8, alpha=0.6, zorder=1)
    for y, colour, name in zip(ys, SERIES, names):
        ax.plot(x, y, color=colour, linewidth=2.0, label=name, zorder=3)
    ax.set_ylim(*ylim)
    ax.set_xlim(-6, 6)
    ax.set_title(title, color=INK, fontsize=12, pad=10)
    ax.set_xlabel(" $x$ ", color=MUTED, fontsize=11)
    ax.tick_params(colors=MUTED, labelsize=9)
    for spine in ("top", "right"):
        ax.spines[spine].set_visible(False)
    for spine in ("left", "bottom"):
        ax.spines[spine].set_color(GRID)

# direct labels, placed where the curves are furthest apart
axes[0].annotate("ReLU", xy=(2.6, 2.6), color=SERIES[2], fontsize=10, weight="bold")
axes[0].annotate("Tanh", xy=(3.2, 1.12), color=SERIES[1], fontsize=10, weight="bold")
axes[0].annotate("Sigmoid", xy=(1.9, 0.30), color=SERIES[0], fontsize=10, weight="bold")

axes[1].annotate("max 0.25", xy=(0, 0.25), xytext=(1.5, 0.45),
    color=MUTED, fontsize=9,
    arrowprops=dict(arrowstyle="->", color=MUTED, linewidth=0.9))

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc="lower center", ncol=3, frameon=False,
    fontsize=10, labelcolor=MUTED, bbox_to_anchor=(0.5, -0.04))

fig.suptitle("Why deep networks moved from sigmoid to ReLU",
    color=INK, fontsize=13.5, weight="bold", y=1.0)

fig.savefig("figures/task4.png", dpi=200, bbox_inches="tight", facecolor=SURFACE)
print("saved figures/task4.png")

```

5 Version Control — GitHub account

limerenceaid -> <https://github.com/limerenceaid>

6 IDE project, pushed to GitHub

Created as a Python project against the Anaconda interpreter from Task 1, committed with git and pushed as a public repository. It holds the three task scripts, the report generator, and every output reproduced above.

<https://github.com/limerenceaid/ELEC576-Assignment0>

Sources

NumPy developers, *NumPy for MATLAB Users*, numpy.org/doc/stable/user/numpy-for-matlab-users.html — the table used in Task 2.

The Matplotlib development team, *Pyplot Tutorial*, matplotlib.org/stable/tutorials/pyplot.html — Tasks 3 and 4.

Anaconda Inc., *Installing on macOS*, docs.anaconda.com/anaconda/install/mac-os/ — Task 1.

The Task 3 script is quoted from the handout; all other code here is my own.